

Java Interview

Questions & Answers – Set 2

Advanced Java, Collections, Streams, Exceptions & OOP

Java 8 / 17 / 21 • Collections Deep Dive • Functional Interfaces

Streams & Lambdas • Comparable & Comparator • String Pool

Exception Hierarchy • Sorting Algorithms • Custom Exceptions

SECTION 1 – Java 8, 17 & 21 Features

1 Java 8 Features

Definition

Java 8 (2014) was the most impactful Java release, introducing functional programming to Java.

- **Lambda Expressions** – Anonymous functions: $(a, b) \rightarrow a + b$
- **Stream API** – Functional pipeline for processing collections (filter, map, reduce).
- **Functional Interfaces** – Interface with one abstract method: Predicate, Function, Consumer, Supplier.
- **Optional** – Wrapper to avoid NullPointerException: Optional.ofNullable().
- **Default & Static Interface Methods** – Methods with body inside interfaces.
- **Method References** – Shortcut: String::toUpperCase, System.out::println.
- **New Date/Time API** – java.time: LocalDate, LocalTime, LocalDateTime, ZonedDateTime.
- **Collectors** – toList(), groupingBy(), joining(), counting().
- **forEach()** – Iterate collections with lambda.
- **String.join(), Map.getOrDefault(), Map.computeIfAbsent()** – Utility improvements.

Example

```
// Lambda
List nums = Arrays.asList(1,2,3,4,5);
nums.stream().filter(n -> n % 2 == 0).forEach(System.out::println); // 2 4

// Optional
Optional name = Optional.ofNullable(null);
System.out.println(name.orElse("Unknown")); // Unknown

// New Date/Time
LocalDate today = LocalDate.now(); // 2025-01-15
LocalDate birthday = LocalDate.of(1995, 6, 20);
long age = ChronoUnit.YEARS.between(birthday, today); // 29
```

2 Java 17 Features (LTS)

Definition

Java 17 (2021) is a Long-Term Support release with many improvements.

- **Sealed Classes** – Restrict which classes can extend/implement: sealed class Shape permits Circle, Rectangle.
- **Pattern Matching for instanceof** – No need to cast after instanceof check.
- **Records** – Concise immutable data classes (auto-generates constructor, getters, equals, hashCode, toString).
- **Text Blocks (finalised)** – Multi-line strings with triple quotes.
- **Switch Expressions (finalised)** – Switch that returns a value using arrow syntax.
- **Enhanced NullPointerExceptions** – More descriptive NPE messages.
- **Removed/Deprecated** – Applet API, Security Manager removed.

Example

```
// Records (Java 17)
record Person(String name, int age) {} // immutable data class
Person p = new Person("Alice", 30);
System.out.println(p.name()); // Alice (auto-getter)

// Sealed Classes
sealed class Shape permits Circle, Rectangle {}
final class Circle extends Shape { double radius; }
final class Rectangle extends Shape { double w, h; }

// Pattern Matching for instanceof
Object obj = "Hello Java 17";
if (obj instanceof String s) { // s is already a String - no cast needed
    System.out.println(s.toUpperCase());
}

// Text Block
String json = """
{"name": "Alice", "age": 30}
""";

// Switch Expression
String day = "MON";
int num = switch(day) {
    case "MON", "TUE" -> 1;
    case "WED" -> 2;
    default -> 0;
};
```

3 Java 21 Features (LTS)

Definition

Java 21 (2023) is the latest LTS release with major concurrency and pattern matching improvements.

- **Virtual Threads (Project Loom)** – Lightweight threads; millions of threads with minimal overhead.
- **Record Patterns** – Deconstruct records in switch/instanceof.
- **Pattern Matching for switch** – Full pattern matching in switch expressions.
- **Sequenced Collections** – New interfaces: SequencedCollection, SequencedMap with first/last methods.
- **String Templates (Preview)** – Embedded expressions in strings.
- **Unnamed Classes & Instance main (Preview)** – Simplified entry point for beginners.

Example

```
// Virtual Threads (Java 21) - lightweight, cheap threads
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 1_000_000; i++) {
        executor.submit(() -> Thread.sleep(Duration.ofSeconds(1)));
    }
} // 1 million virtual threads without memory crash!

// Pattern Matching in switch (Java 21)
Object shape = new Circle(5.0);
String desc = switch (shape) {
    case Circle c -> "Circle with radius " + c.radius();
    case Rectangle r -> "Rectangle " + r.w() + "x" + r.h();
    default -> "Unknown shape";
};

// Sequenced Collections
SequencedCollection list = new ArrayList<>(List.of("a","b","c"));
list.getFirst(); // a
list.getLast(); // c
list.reversed(); // [c, b, a]
```

SECTION 2 – Collections Deep Dive

4 All Java Collections – When to use List, Set, Map, Queue

Definition

Java Collections Framework provides data structures for every use case.

- **List** – Ordered, allows duplicates, index-based access. Use when order matters.
- **Set** – No duplicates. Use when uniqueness is required.
- **Map** – Key-value pairs, unique keys. Use for lookups by key.
- **Queue/Deque** – FIFO/LIFO processing. Use for task queues, undo operations.

Example

```
// LIST - ordered, duplicates allowed
List list = new ArrayList<>(); // fast random access O(1)
List link = new LinkedList<>(); // fast insert/delete O(1)

// SET - no duplicates
Set hash = new HashSet<>(); // O(1) ops, no order
Set linked = new LinkedHashSet<>(); // insertion order
Set tree = new TreeSet<>(); // sorted order

// MAP - key-value, unique keys
Map map = new HashMap<>(); // O(1) ops
Map linked = new LinkedHashMap<>(); // insertion order
Map sorted = new TreeMap<>(); // sorted by key
Map conc = new ConcurrentHashMap<>(); // thread-safe

// QUEUE / DEQUE
Queue queue = new LinkedList<>(); // FIFO
Deque deque = new ArrayDeque<>(); // stack or queue
Queue pq = new PriorityQueue<>(); // min-heap
```

5 ArrayList vs LinkedList – when to use each

Definition

Both implement List interface but have very different performance characteristics.

Example

```
// ArrayList – backed by dynamic array
// O(1) get(index) – use when you READ a lot
List al = new ArrayList<>();
al.add("a"); al.add("b");
al.get(0); // O(1) – fast
al.add(0, "first"); // O(n) – slow (shifts elements)

// LinkedList – doubly linked list
// O(1) add/remove at ends – use when you INSERT/DELETE a lot
List ll = new LinkedList<>();
ll.addFirst("first"); // O(1) – fast
ll.addLast("last"); // O(1) – fast
ll.get(5); // O(n) – slow (traverse to index)
```

Note: Rule: Use ArrayList for most cases. Use LinkedList only when you do frequent insertions/deletions at the beginning.

6 How HashMap works internally

Definition

HashMap uses an array of buckets with linked lists (or trees for Java 8+) to store key-value pairs.

- **Step 1 – Hash:** key.hashCode() is computed and hashed to a bucket index.
- **Step 2 – Bucket:** The entry is placed in the bucket (array slot) at that index.
- **Step 3 – Collision:** If two keys hash to the same bucket, they form a linked list.
- **Step 4 – Tree:** When a bucket exceeds 8 entries, it converts to a Red-Black tree (Java 8+).
- **Load Factor:** Default 0.75; when 75% full, the array resizes (doubles).

Example

```
Map map = new HashMap<>();
map.put("Alice", 90); // hash("Alice") -> bucket index -> store
map.put("Bob", 85); // hash("Bob") -> different bucket

// Same key added again -> OVERRIDES existing value
map.put("Alice", 95); // Alice: 90 -> 95 (replaced)
System.out.println(map.get("Alice")); // 95

// Key collision (same hash, different key)
// Both stored in same bucket as linked list entries
// Resolved via equals() method

// RULE: If you override equals(), you MUST override hashCode()
```

7 What happens when duplicate key is added to Map?

Definition

HashMap replaces the old value with the new value when the same key is inserted again.

Example

```
Map map = new HashMap<>();

// 1st entry
map.put(101, "Reshmi");
// map: {101=Reshmi}

// 2nd entry - SAME key 101 with different value
map.put(101, "Nikhitha");
// map: {101=Nikhitha} <-- Reshmi is OVERWRITTEN

// 3rd entry - DIFFERENT key 102
map.put(102, "Nikhitha");
// map: {101=Nikhitha, 102=Nikhitha} <-- both co-exist (different keys)

// To ADD without overriding, use putIfAbsent:
map.putIfAbsent(101, "Likhitha");
// map: {101=Nikhitha} <-- NOT changed because key 101 already exists

// To keep multiple values per key, use Map>:
Map> multiMap = new HashMap<>();
multiMap.computeIfAbsent(101, k -> new ArrayList<>()).add("Reshmi");
multiMap.computeIfAbsent(101, k -> new ArrayList<>()).add("Nikhitha");
// map: {101=[Reshmi, Nikhitha]}
```

Note: Key rule: HashMap only stores ONE value per key. Same key = override. Different key = separate entry.

8 Comparable vs Comparator

Definition

Both are used for sorting objects but differ in where the comparison logic lives.

- **Comparable** – The class itself defines the natural sort order. Implements `compareTo()`. Modifies the class. Used by `Collections.sort()`.
- **Comparator** – External class defines sort order. Implements `compare()`. Does NOT modify the class. Allows multiple different orderings.

Example

```
// Comparable - natural ordering (inside the class)
class Employee implements Comparable {
    int id; String name; double salary;
    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id); // sort by id
    }
}

List list = new ArrayList<>();
Collections.sort(list); // uses compareTo automatically

// Comparator - custom ordering (outside the class)
// Sort by salary descending
Comparator bySalaryDesc =
    Comparator.comparingDouble(Employee::getSalary).reversed();

// Sort by name
Comparator byName =
    Comparator.comparing(Employee::getName);

list.sort(bySalaryDesc); // sort by salary desc
list.sort(byName); // sort by name

// Chain comparators
list.sort(Comparator.comparing(Employee::getDept)
    .thenComparing(Employee::getSalary));
```

Note: Use Comparable for the default natural order. Use Comparator for custom or multiple sort criteria.

Aspect	Comparable	Comparator
Package	java.lang	java.util
Method	<code>compareTo(T o)</code>	<code>compare(T o1, T o2)</code>
Modifies class	Yes	No
Multiple orders	Only one natural order	Can define many
Usage	<code>Collections.sort(list)</code>	<code>list.sort(comparator)</code>

9 Sorting Algorithms: BubbleSort and MergeSort

Definition

Two fundamental sorting algorithms with very different performance characteristics.

Example

```
// BUBBLE SORT -  $O(n^2)$  - simple but slow
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j+1] = temp;
            }
        }
    }
}

// Input: [64, 34, 25, 12, 22]
// Output: [12, 22, 25, 34, 64]

// MERGE SORT -  $O(n \log n)$  - divide and conquer
public static void mergeSort(int[] arr, int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid); // sort left half
        mergeSort(arr, mid + 1, right); // sort right half
        merge(arr, left, mid, right); // merge both halves
    }
}

private static void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int[] L = new int[n1], R = new int[n2];
    System.arraycopy(arr, left, L, 0, n1);
    System.arraycopy(arr, mid+1, R, 0, n2);
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

Note: BubbleSort: simple, $O(n^2)$, for small data. MergeSort: efficient, $O(n \log n)$, stable, for large data.

SECTION 3 – Functional Interfaces, Lambdas & Streams

10 What is a Functional Interface?

Definition

A functional interface is an interface with exactly ONE abstract method. It can be implemented using a lambda expression or method reference. The `@FunctionalInterface` annotation enforces this rule at compile time.

Example

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b); // exactly ONE abstract method
    // Can have default and static methods
    default void print() { System.out.println("Calculator"); }
}

// Implement with lambda
Calculator add = (a, b) -> a + b;
Calculator mul = (a, b) -> a * b;
System.out.println(add.calculate(3, 5)); // 8
System.out.println(mul.calculate(3, 5)); // 15
```

11 Built-in Functional Interfaces in Java 8

Definition

Java 8 provides ready-to-use functional interfaces in `java.util.function` package.

- **Predicate<T>** – Takes T, returns boolean. `test(T t)`. Used for filtering.
- **Function<T,R>** – Takes T, returns R. `apply(T t)`. Used for transformation.
- **Consumer<T>** – Takes T, returns nothing. `accept(T t)`. Used for side effects.
- **Supplier<T>** – Takes nothing, returns T. `get()`. Used for lazy values.
- **BiFunction<T,U,R>** – Takes T and U, returns R.
- **BiPredicate<T,U>** – Takes T and U, returns boolean.
- **UnaryOperator<T>** – Takes T, returns T (extends `Function<T,T>`).
- **BinaryOperator<T>** – Takes two T, returns T.

Example

```
// Predicate - filter condition
Predicate isEven = n -> n % 2 == 0;
System.out.println(isEven.test(4)); // true
Predicate notEmpty = s -> !s.isEmpty();

// Function - transform value
Function strLen = String::length;
System.out.println(strLen.apply("Hello")); // 5

// Consumer - side effect (no return)
Consumer printer = System.out::println;
printer.accept("Hello"); // prints: Hello

// Supplier - lazy value provider
Supplier greeting = () -> "Good Morning!";
System.out.println(greeting.get()); // Good Morning!

// In Streams
List names = List.of("Alice", "Bob", "Charlie", "");
names.stream()
    .filter(notEmpty) // Predicate
    .map(strLen) // Function
    .forEach(printer); // Consumer
// 5, 3, 7
```

12 Lambda Expression – What and How

Definition

A lambda expression is a concise way to represent an anonymous function. It implements a functional interface inline without creating a separate class.

Example

```
// Syntax: (parameters) -> expression OR (parameters) -> { body }

// No parameters
Runnable r = () -> System.out.println("Running!");
r.run(); // Running!

// One parameter (parentheses optional)
Consumer print = s -> System.out.println(s);
print.accept("Hello"); // Hello

// Multiple parameters
Comparator comp = (a, b) -> a - b;

// Block body with return
Function classify = n -> {
    if (n > 0) return "Positive";
    else if (n < 0) return "Negative";
    else return "Zero";
};
System.out.println(classify.apply(-5)); // Negative

// In Collections
List fruits = new ArrayList<>(Arrays.asList("Mango", "Apple", "Banana"));
fruits.sort((f1, f2) -> f1.compareTo(f2)); // alphabetical
fruits.forEach(f -> System.out.println(f));
```

13 map() vs flatMap() in Streams

Definition

Both transform stream elements but differ in how they handle nested structures.

- **map()** – Transforms each element to ONE other element. Returns Stream<T>.
- **flatMap()** – Transforms each element to ZERO OR MORE elements and flattens the result. Returns Stream<T> (not Stream<Stream<T>>).

Example

```
// map() - one-to-one transformation
List names = List.of("Alice", "Bob", "Charlie");
List lengths = names.stream()
    .map(String::length) // Alice->5, Bob->3, Charlie->7
    .collect(Collectors.toList());
// [5, 3, 7]

// flatMap() - one-to-many, then flatten
List> nestedList = List.of(
    List.of(1, 2, 3),
    List.of(4, 5),
    List.of(6, 7, 8, 9)
);
List flat = nestedList.stream()
    .flatMap(Collection::stream) // flatten nested lists
    .collect(Collectors.toList());
// [1, 2, 3, 4, 5, 6, 7, 8, 9]

// flatMap() with sentences
List sentences = List.of("Hello World", "Java Streams");
List words = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .collect(Collectors.toList());
// [Hello, World, Java, Streams]
```

Note: Use map() for one-to-one. Use flatMap() when each element produces multiple elements or is itself a collection/array.

14 **forEach – Intermediate or Terminal?**

Definition

`forEach()` is a TERMINAL operation. It triggers the stream pipeline execution and processes each element. After `forEach()`, you cannot chain more stream operations.

Example

```
// forEach() is TERMINAL
List names = List.of("Alice", "Bob", "Charlie");

names.stream()
  .filter(n -> n.length() > 3) // intermediate
  .map(String::toUpperCase) // intermediate
  .forEach(System.out::println); // TERMINAL - triggers execution
// Output: ALICE, CHARLIE

// Cannot chain after forEach - it returns void
// names.stream().forEach(...).filter(...) -- COMPILE ERROR

// peek() is similar but INTERMEDIATE (for debugging)
names.stream()
  .peek(n -> System.out.println("Before: " + n)) // intermediate
  .filter(n -> n.length() > 3)
  .peek(n -> System.out.println("After: " + n)) // intermediate
  .count(); // terminal
```

15 All Stream Operations – Intermediate vs Terminal

Definition

Intermediate operations return a new Stream and are lazy. Terminal operations trigger execution.

Example

```
// INTERMEDIATE (lazy - don't execute until terminal):
// filter(Predicate) - keep elements matching condition
// map(Function) - transform each element
// flatMap(Function) - flatten nested structures
// sorted() - sort elements
// sorted(Comparator) - sort with custom comparator
// distinct() - remove duplicates
// limit(n) - take first n elements
// skip(n) - skip first n elements
// peek(Consumer) - debug; same as forEach but intermediate
// mapToInt/Long/Double - convert to primitive streams

// TERMINAL (eager - trigger execution, return non-stream):
// collect(Collector) - gather into List/Set/Map
// forEach(Consumer) - process each element
// count() - count elements
// findFirst() - first element (Optional)
// findAny() - any element (Optional)
// anyMatch(Predicate) - true if any element matches
// allMatch(Predicate) - true if all elements match
// noneMatch(Predicate) - true if no elements match
// min(Comparator) - minimum element (Optional)
// max(Comparator) - maximum element (Optional)
// reduce(BinaryOp) - aggregate to single value
// toList() - Java 16+: shorthand for collect(toList())
// toArray() - convert to array
```

16 equals() and hashCode() – Contract and Rules

Definition

equals() and hashCode() must be overridden together and follow a strict contract.

- **Contract Rule 1:** If a.equals(b) is TRUE, then a.hashCode() MUST equal b.hashCode().
- **Contract Rule 2:** If a.hashCode() equals b.hashCode(), a.equals(b) MAY or MAY NOT be true (hash collision is allowed).
- **Contract Rule 3:** If a.equals(b) is FALSE, their hashCodes CAN be the same (collision) or different.
- **Breaking the contract:** Objects stored in HashMap/HashSet behave incorrectly if you override one without the other.

Example

```
// WRONG: Only override equals without hashCode
class BadKey {
    int id;
    @Override
    public boolean equals(Object o) { return ((BadKey)o).id == this.id; }
    // NO hashCode override!
}

// BadKey with id=1 and another BadKey with id=1 are 'equal'
// but stored in DIFFERENT HashMap buckets -> put/get breaks!

// CORRECT: Override both equals and hashCode
class Employee {
    int id; String name;
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Employee)) return false;
        return this.id == ((Employee)o).id;
    }
    @Override
    public int hashCode() {
        return Objects.hash(id); // use id for hash
    }
}

// Q: Two strings equal by hashCode – equal by equals?
// A: NOT necessarily (hash collision possible)

// Q: Two strings equal by equals – same hashCode?
// A: YES – this MUST be true per the contract
String s1 = "abc", s2 = "abc";
s1.equals(s2); // true
s1.hashCode() == s2.hashCode(); // MUST be true
```


17 equals() vs == operator

Definition

Both compare objects but in fundamentally different ways.

- **== operator** – Compares REFERENCE (memory address). Two variables point to the same object?
- **equals() method** – Compares CONTENT (logical equality). Do two objects have the same data?
- **For primitives:** == compares VALUES (int a == int b).
- **String pool:** String literals are pooled, so == may work, but always use equals() for Strings.

Example

```
// String comparison
String a = "Hello"; // from string pool
String b = "Hello"; // same pool object
String c = new String("Hello"); // new heap object

System.out.println(a == b); // true (same pool reference)
System.out.println(a == c); // false (different object)
System.out.println(a.equals(c)); // true (same content)

// Objects
Employee e1 = new Employee(1, "Alice");
Employee e2 = new Employee(1, "Alice");
System.out.println(e1 == e2); // false (different objects)
System.out.println(e1.equals(e2)); // true (if equals is overridden)

// RULE: Always use equals() for String and Object comparison, never ==
```

18 What is record in Java 17?

Definition

A record is a concise way to declare an immutable data-carrying class. Java auto-generates the constructor, getters, equals(), hashCode(), and toString().

Example

```
// Traditional class - verbose
class PersonOld {
    private final String name;
    private final int age;
    public PersonOld(String n, int a) { this.name=n; this.age=a; }
    public String getName() { return name; }
    public int getAge() { return age; }
    @Override public String toString() { return "Person(" + name + "," + age + ")"; }
    @Override public boolean equals(Object o) { /* boilerplate */ }
    @Override public int hashCode() { /* boilerplate */ }
}

// Record - concise (Java 16+)
record Person(String name, int age) {}

// Usage
Person p = new Person("Alice", 30);
System.out.println(p.name()); // Alice (accessor, not getName())
System.out.println(p.age()); // 30
System.out.println(p); // Person[name=Alice, age=30]

// Records are immutable - no setters
// p.name = "Bob"; // COMPILER ERROR

// Records can have custom methods
record Circle(double radius) {
    double area() { return Math.PI * radius * radius; }
}
```

19 Predicate, Function, Consumer and Supplier – Examples

Definition

The four core functional interfaces in Java 8, each serving a specific purpose.

Example

```
// PREDICATE: T -> boolean
// Use for: filtering, conditional checks
Predicate isLong = s -> s.length() > 5;
Predicate isPos = n -> n > 0;
// Compose predicates
Predicate isLongAndUpper = isLong.and(s -> s.equals(s.toUpperCase()));
isLong.test("Hello World"); // true
isLong.negate().test("Hi"); // true (negated)

// FUNCTION: T -> R
// Use for: transformation, conversion
Function len = String::length;
Function str = Object::toString;
Function upper = String::toUpperCase;
// Chain functions
Function lenOfUpper = upper.andThen(len);
lenOfUpper.apply("hello"); // 5

// CONSUMER: T -> void
// Use for: side effects (logging, saving, printing)
Consumer log = s -> System.out.println("LOG: " + s);
Consumer save = emp -> empRepo.save(emp);
Consumer logAndSave = log.andThen(s -> db.save(s));

// SUPPLIER: () -> T
// Use for: lazy initialization, factory methods
Supplier today = LocalDate::now;
Supplier emptyList = ArrayList::new;
Supplier emp = () -> new Employee("Bob", 50000);
today.get(); // returns today's date
```

20 filter() in Streams – Use and Examples

Definition

filter() is an intermediate stream operation that keeps only elements matching a Predicate.

Example

```
List employees = List.of(
    new Employee("Alice", "IT", 80000),
    new Employee("Bob", "HR", 60000),
    new Employee("Carol", "IT", 90000),
    new Employee("Dave", "Finance", 70000)
);

// Filter by department
List itTeam = employees.stream()
    .filter(e -> e.getDept().equals("IT"))
    .collect(Collectors.toList());
// [Alice(IT), Carol(IT)]

// Filter by salary range
List highEarners = employees.stream()
    .filter(e -> e.getSalary() > 70000)
    .collect(Collectors.toList());
// [Alice, Carol]

// Multiple filters (chained)
List names = employees.stream()
    .filter(e -> e.getDept().equals("IT"))
    .filter(e -> e.getSalary() > 75000)
    .map(Employee::getName)
    .collect(Collectors.toList());
// [Alice, Carol] -- Carol too since 90000 > 75000

// Predicate composition
Predicate isIT = e -> e.getDept().equals("IT");
Predicate isHighPay = e -> e.getSalary() > 75000;
employees.stream().filter(isIT.and(isHighPay)).collect(toList());
```

SECTION 4 – Strings, Immutability & String Pool

21 How String is immutable – with explanation

Definition

A String object in Java cannot be changed after it is created. Any operation that seems to modify a String actually creates a NEW String object.

Example

```
// String is stored in String Constant Pool (heap)
String s1 = "Hello"; // creates "Hello" in pool
String s2 = "Hello"; // reuses SAME pool object
System.out.println(s1 == s2); // true - same reference

// 'Modifying' creates a NEW object
String s3 = s1.concat(" World");
System.out.println(s1); // Hello (UNCHANGED)
System.out.println(s3); // Hello World (new object)

// Reassigning s1 points to a new object, doesn't change 'Hello'
s1 = s1 + " Java";
System.out.println(s1); // Hello Java (new object)
// 'Hello' still exists in pool but s1 no longer points to it

// WHY immutable?
// 1. Security - String used as HashMap key, class name, URL
// 2. Thread safety - immutable objects are inherently thread-safe
// 3. Caching - same literal can be shared via pool
```

22 String Constant Pool – Explanation

Definition

The String Constant Pool (or String Pool) is a special memory area in the Java Heap that stores String literals to avoid creating duplicate String objects.

Example

```
// String literal - stored in pool
String a = "Java"; // checks pool, creates if not exists
String b = "Java"; // reuses existing pool object
System.out.println(a == b); // true - same pool reference

// new String() - bypasses pool, creates on heap
String c = new String("Java"); // ALWAYS creates new object
System.out.println(a == c); // false - different objects
System.out.println(a.equals(c)); // true - same content

// intern() - forces string into pool
String d = c.intern(); // moves to pool if not already there
System.out.println(a == d); // true - d now references pool object

// Java 7+: String pool is in the Heap (moved from PermGen)
// Java 8: PermGen removed, pool is in regular Heap
```

23 String vs StringBuilder vs StringBuffer

Definition

Three ways to work with text in Java, each with different mutability and thread-safety.

Example

```
// String - immutable, thread-safe, in string pool
String s = "Hello";
s += " World"; // creates new object each time - slow in loops

// StringBuilder - mutable, NOT thread-safe, fast
// Use when: building strings in a loop (single thread)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) {
    sb.append("item").append(i).append(",");
}
String result = sb.toString(); // item0,item1,item2,item3,item4,
sb.insert(0, "[");
sb.delete(sb.length()-1, sb.length()); // remove last comma
sb.reverse();

// StringBuffer - mutable, thread-safe, slower than StringBuilder
// Use when: building strings in multi-threaded environment
StringBuffer sbf = new StringBuffer();
sbf.append("thread-safe");
```

Note: In most cases use `StringBuilder`. Use `StringBuffer` only when multiple threads modify the same string.

Feature	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes	No	Yes
Performance	Slow (new objects)	Fast	Moderate
Use When	Fixed strings	Single-thread concat	Multi-thread concat

SECTION 5 – Exception Handling in Depth

24 Exception Hierarchy in Java

Definition

All exceptions and errors in Java extend from the Throwable class.

Example

```
// Complete Exception Hierarchy
java.lang.Throwable
|
+-- java.lang.Error (serious problems, don't catch)
|   +-- OutOfMemoryError
|   +-- StackOverflowError
|   +-- VirtualMachineError
|
+-- java.lang.Exception (recoverable problems)
|
+-- RuntimeException (UNCHECKED – not required to handle)
|   +-- NullPointerException
|   +-- ArrayIndexOutOfBoundsException
|   +-- ClassCastException
|   +-- ArithmeticException (divide by zero)
|   +-- IllegalArgumentException
|   +-- NumberFormatException
|
+-- IOException (CHECKED – must handle)
+-- SQLException
+-- ClassNotFoundException
+-- FileNotFoundException
+-- ParseException
```


25 Checked vs Unchecked Exceptions

Definition

Java exceptions are classified by whether the compiler forces you to handle them.

- **Checked (Compile-time):** Extends Exception (not RuntimeException). Compiler forces you to handle with try-catch or declare with throws. Examples: IOException, SQLException.
- **Unchecked (Runtime):** Extends RuntimeException. Compiler does NOT require handling. Examples: NullPointerException, ArrayIndexOutOfBoundsException.

Example

```
// CHECKED - must handle at compile time
public void readFile(String path) {
    try {
        FileReader fr = new FileReader(path); // IOException (checked)
    } catch (FileNotFoundException e) {
        System.out.println("File not found: " + e.getMessage());
    } catch (IOException e) {
        System.out.println("IO Error: " + e.getMessage());
    }
}

// OR declare with throws (let caller handle)
public void readFile(String path) throws IOException {
    FileReader fr = new FileReader(path);
}

// UNCHECKED - optional handling
String s = null;
s.length(); // NullPointerException - no forced handling

// How to handle unchecked:
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

26 throw vs throws – Complete Explanation

Definition

Two very different keywords for exception handling.

- **throw** – Used INSIDE a method to actually throw (trigger) an exception. Followed by an exception object.
- **throws** – Used in METHOD SIGNATURE to declare that the method might throw a checked exception. Lets the caller handle it.

Example

```
// throws – declares what can be thrown (in method signature)
public void connectDB(String url) throws SQLException, IOException {
    if (url == null)
        throw new IllegalArgumentException("URL cannot be null"); // throw
    // ... connection logic
}

// Handling with throws – can you catch it elsewhere?
// YES! The caller must handle it or re-declare throws
public void startApp() {
    try {
        connectDB("jdbc:mysql://..."); // handled HERE
    } catch (SQLException e) {
        System.out.println("DB error: " + e.getMessage());
    } catch (IOException e) {
        System.out.println("IO error: " + e.getMessage());
    }
}

// OR re-declare and let FURTHER caller handle
public void startApp() throws SQLException, IOException {
    connectDB("jdbc:mysql://..."); // not handled, propagated up
}
```

27 Custom Exceptions – How to Create

Definition

You can create your own exception classes by extending `Exception` (checked) or `RuntimeException` (unchecked).

Example

```
// Custom CHECKED exception
public class DatabaseException extends Exception {
    public DatabaseException(String message) {
        super(message);
    }
    public DatabaseException(String message, Throwable cause) {
        super(message, cause);
    }
}

// Custom UNCHECKED exception
public class StudentNotFoundException extends RuntimeException {
    private final Long studentId;
    public StudentNotFoundException(Long id) {
        super("Student not found with id: " + id);
        this.studentId = id;
    }
    public Long getStudentId() { return studentId; }
}

// Usage
@Service
public class StudentService {
    public Student findById(Long id) {
        return studentRepo.findById(id)
            .orElseThrow(() -> new StudentNotFoundException(id));
        // Throws: Student not found with id: 42
    }
}

// In Spring Boot: handled by @ControllerAdvice
@ExceptionHandler(StudentNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public ErrorResponse handle(StudentNotFoundException ex) {
    return new ErrorResponse(404, ex.getMessage());
}
```

28 try-catch-finally and when to use finally

Definition

The finally block always executes regardless of whether an exception occurred.

Example

```
// try-catch-finally structure
public void readData(String filePath) {
    FileReader fr = null;
    try {
        fr = new FileReader(filePath);
        // ... read data
        int result = 10 / 2; // no exception
    } catch (FileNotFoundException e) {
        System.out.println("File not found!");
    } catch (ArithmeticException e) {
        System.out.println("Math error!");
    } catch (Exception e) {
        System.out.println("General error: " + e.getMessage());
    } finally {
        // ALWAYS runs - even if exception, even if return in try
        if (fr != null) {
            try { fr.close(); } catch (IOException e) {}
        }
        System.out.println("Cleanup done");
    }
}

// Modern way: try-with-resources (auto-closes)
try (FileReader fr = new FileReader(filePath);
    BufferedReader br = new BufferedReader(fr)) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} // fr and br automatically closed - no finally needed!

// Resource must implement AutoCloseable
```

Note: Use finally for resource cleanup. Prefer try-with-resources (Java 7+) for Closeable resources.

29 Checked Exception in your application – what to do

Definition

When your code throws a checked exception, you have two choices.

Example

```
// OPTION 1: Handle it (try-catch) – use when you can recover
public String readConfig(String path) {
    try {
        return Files.readString(Path.of(path));
    } catch (IOException e) {
        log.error("Config file not found: {}", path);
        return "{}" // return default config
    }
}

// OPTION 2: Propagate with throws – use when caller should decide
public String readConfig(String path) throws IOException {
    return Files.readString(Path.of(path));
}

// OPTION 3: Wrap in unchecked – common in Spring apps
public String readConfig(String path) {
    try {
        return Files.readString(Path.of(path));
    } catch (IOException e) {
        throw new RuntimeException("Failed to read config: " + path, e);
        // or throw custom: throw new ConfigLoadException(path, e);
    }
}

// In Spring Boot: @ControllerAdvice catches and handles exceptions globally
```

SECTION 6 – Programs & Data Structures

30 Count occurrences of each character in a String

Definition

Count how many times each character appears in a given string.

Example

```
// Method 1: HashMap (complete character frequency)
public static void countChars(String input) {
    Map freq = new LinkedHashMap<>();
    for (char c : input.toCharArray()) {
        freq.put(c, freq.getOrDefault(c, 0) + 1);
    }
    freq.forEach((k, v) -> System.out.println(k + " -> " + v));
}

countChars("programming");
// p->1, r->2, o->1, g->2, a->1, m->2, i->1, n->1

// Method 2: Java 8 Streams
String s = "programming";
Map result = s.chars()
    .mapToObj(c -> (char)c)
    .collect(Collectors.groupingBy(c -> c, Collectors.counting()));
System.out.println(result);

// Method 3: Count specific character
String text = "banana";
char target = 'a';
long count = text.chars().filter(c -> c == target).count();
System.out.println("'a' appears " + count + " times"); // 3
```

31 Check if a String is an Anagram

Definition

Two strings are anagrams if they have the same characters in any order.

Example

```
// Method 1: Sort and compare (simple)
public static boolean isAnagram(String s1, String s2) {
    if (s1 == null || s2 == null || s1.length() != s2.length())
        return false;
    char[] arr1 = s1.toLowerCase().toCharArray();
    char[] arr2 = s2.toLowerCase().toCharArray();
    Arrays.sort(arr1);
    Arrays.sort(arr2);
    return Arrays.equals(arr1, arr2);
}

// Method 2: Frequency map (efficient)
public static boolean isAnagramFast(String s1, String s2) {
    if (s1.length() != s2.length()) return false;
    int[] freq = new int[256];
    for (char c : s1.toLowerCase().toCharArray()) freq[c]++;
    for (char c : s2.toLowerCase().toCharArray()) freq[c]--;
    for (int f : freq) if (f != 0) return false;
    return true;
}

System.out.println(isAnagram("listen", "silent")); // true
System.out.println(isAnagram("Triangle", "Integral")); // true
System.out.println(isAnagram("hello", "world")); // false
```

32 Can we override static methods? (Class and Interface)

Definition

No. Static methods cannot be overridden in Java. They can only be HIDDEN.

- **Static method hiding (class):** A subclass can declare a static method with the same name. This HIDES the parent's method; it does NOT override it.
- **Interface static methods:** Cannot be overridden or inherited by implementing classes at all.
- **Why?** Method overriding uses dynamic dispatch (runtime polymorphism). Static methods are resolved at COMPILE TIME based on the reference type.

Example

```
// Static method hiding in classes
class Parent {
    static void greet() { System.out.println("Parent greet"); }
    void display() { System.out.println("Parent display"); }
}
class Child extends Parent {
    static void greet() { System.out.println("Child greet"); } // HIDING
    @Override
    void display() { System.out.println("Child display"); } // OVERRIDE
}

Parent p = new Child();
p.greet(); // Parent greet (resolved at compile time - hiding)
p.display(); // Child display (resolved at runtime - overriding)

// Interface static methods - NOT inherited
interface MyInterface {
    static void helper() { System.out.println("Interface helper"); }
}
class MyClass implements MyInterface {
    // Cannot override helper()
    // MyClass.helper() - compile error (not accessible via class)
    // Must call: MyInterface.helper()
}
```

Note: @Override on a static method causes a COMPILE ERROR. Static methods are hidden, not overridden.

33 What is default method in Interface?

Definition

A default method is a method with an implementation inside an interface (Java 8+). It allows adding new methods to interfaces without breaking existing implementations.

Example

```
interface Vehicle {
    void start(); // abstract - must implement

    // Default method - optional to override
    default void stop() {
        System.out.println("Vehicle stopped");
    }

    // Static method - called on interface directly
    static Vehicle create() { return new Car(); }
}

class Car implements Vehicle {
    @Override
    public void start() { System.out.println("Car started"); }
    // stop() is inherited from interface default
}

class Truck implements Vehicle {
    @Override
    public void start() { System.out.println("Truck started"); }
    @Override
    public void stop() { System.out.println("Truck stopped loudly"); }
    // Overrides the default stop()
}

Car car = new Car();
car.start(); // Car started
car.stop(); // Vehicle stopped (from interface default)
```

Note: Default methods were added to allow interface evolution without breaking backward compatibility.

QUICK REFERENCE TABLES

Java 8 / 17 / 21 Feature Summary

Version	Key Features
Java 8 (2014)	Lambda, Streams, Optional, Functional Interfaces, Default methods, Date/Time API, Method References
Java 11 (LTS)	var in lambda, HTTP Client API, String methods: isBlank(), strip(), lines()
Java 17 (LTS)	Sealed Classes, Records, Pattern Matching instanceof, Text Blocks, Switch Expressions
Java 21 (LTS)	Virtual Threads, Pattern Matching switch, Sequenced Collections, Record Patterns

Functional Interface Quick Reference

Interface	Input	Output	Method	Example Use
Predicate<T>	T	boolean	test(T)	filter(n -> n>0)
Function<T,R>	T	R	apply(T)	map(String::length)
Consumer<T>	T	void	accept(T)	forEach(System.out::println)
Supplier<T>	nothing	T	get()	orElseGet(() -> new Obj())
BiFunction<T,U,R>	T,U	R	apply(T,U)	merge, combine two values
UnaryOperator<T>	T	T	apply(T)	map(s -> s.toUpperCase())
BinaryOperator<T>	T,T	T	apply(T,T)	reduce((a,b) -> a+b)

Collection Comparison

Collection	Order	Duplicates	Null	Thread-safe	Time Complexity
ArrayList	Insertion	Yes	Yes	No	get O(1), add O(1)
LinkedList	Insertion	Yes	Yes	No	get O(n), add O(1)
HashSet	None	No	One	No	add/get O(1)
LinkedHashSet	Insertion	No	One	No	add/get O(1)
TreeSet	Sorted	No	No	No	add/get O(log n)
HashMap	None	V:Yes	One K	No	get/put O(1)
LinkedHashMap	Insertion	V:Yes	One K	No	get/put O(1)
TreeMap	Key sorted	V:Yes	No K	No	get/put O(log n)
ConcurrentHashMap	None	V:Yes	No K/V	Yes	get/put O(1)

Exception Types Quick Reference

Type	Extends	Must Handle?	Examples
Checked	Exception	YES (compile error)	IOException, SQLException, ClassNotFoundException

Type	Extends	Must Handle?	Examples
Unchecked	RuntimeException	NO (optional)	NullPointerException, IndexOutOfBoundsException, ClassCastException
Error	Error	NO (don't catch)	OutOfMemoryError, StackOverflowError